

Web Security Fundamentals - Testing & Audit Report

Author: Amro Zaid Al Mohaisen

Date: August 11, 2026

Training Module: Task 3 - Web Security Fundamentals

Organization: Batman Technology - Amman, Jordan

1. Executive Summary

This security report documents the implementation, hardening, and test verification performed on the E-Commerce REST API backend. The application was reviewed against web security standards to resolve fundamental vulnerabilities, enforce robust access control mechanisms, and protect sensitive database assets connected to Neon PostgreSQL.

Key Security Implementations:

- **Secrets & Environment Management:** Isolated all connection strings, API keys, and secret credentials inside .env (excluded from git tracking via .gitignore).
- **SQL Injection Prevention:** Refactored database queries across all controllers to use parameterized placeholders (\$1, \$2).
- **Password Hashing & Sensitive Data Protection:** Applied bcrypt with 12 salt rounds for password encryption and excluded password_hash fields from all outgoing JSON responses.
- **Authentication & Role-Based Access Control (RBAC):** Implemented stateless JWT bearer authentication along with authorize middleware to restrict administrative endpoints.
- **IDOR Protection:** Added ownership verification middleware (checkOwnership) to prevent horizontal privilege escalation across user resources.
- **Rate Limiting & Security Headers:** Applied helmet security headers, restricted cors origins, and rate-limited brute-force targets (/api/auth/login) using express-rate-limit.

2. Initial Security Audit Summary (SECURITY_REVIEW.md)

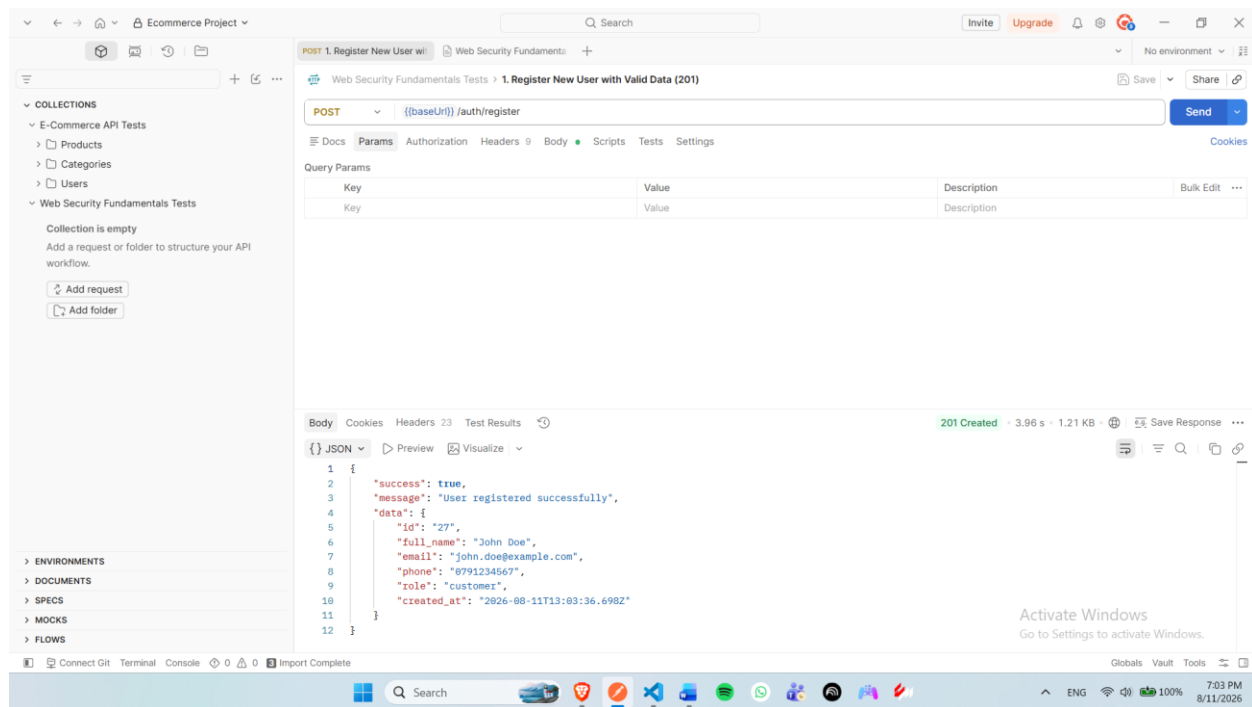
Prior to remediation, the codebase was audited for fundamental security flaws:

#	Vulnerability / Issue	Location	Severity	Mitigation Implemented
1	Plaintext Passwords / Weak Hashing	authController.js	Critical	Replaced with bcrypt.hash() (12 rounds).
2	SQL Injection Risk	Controllers	Critical	Converted string concatenation to parameterized queries (\$1, \$2).
3	Unprotected Administrative Routes	productsRoutes.js	High	Protected routes with authenticate & authorize('admin') middleware.
4	Sensitive Data Leakage (password_hash)	User Queries	High	Explicitly excluded password hashes from SQL SELECT statements.
5	Exposure of Stack Traces / Internal Errors	app.js	Medium	Created safe centralized error handling without stack trace output.

3. Mandatory Postman Security Test Results

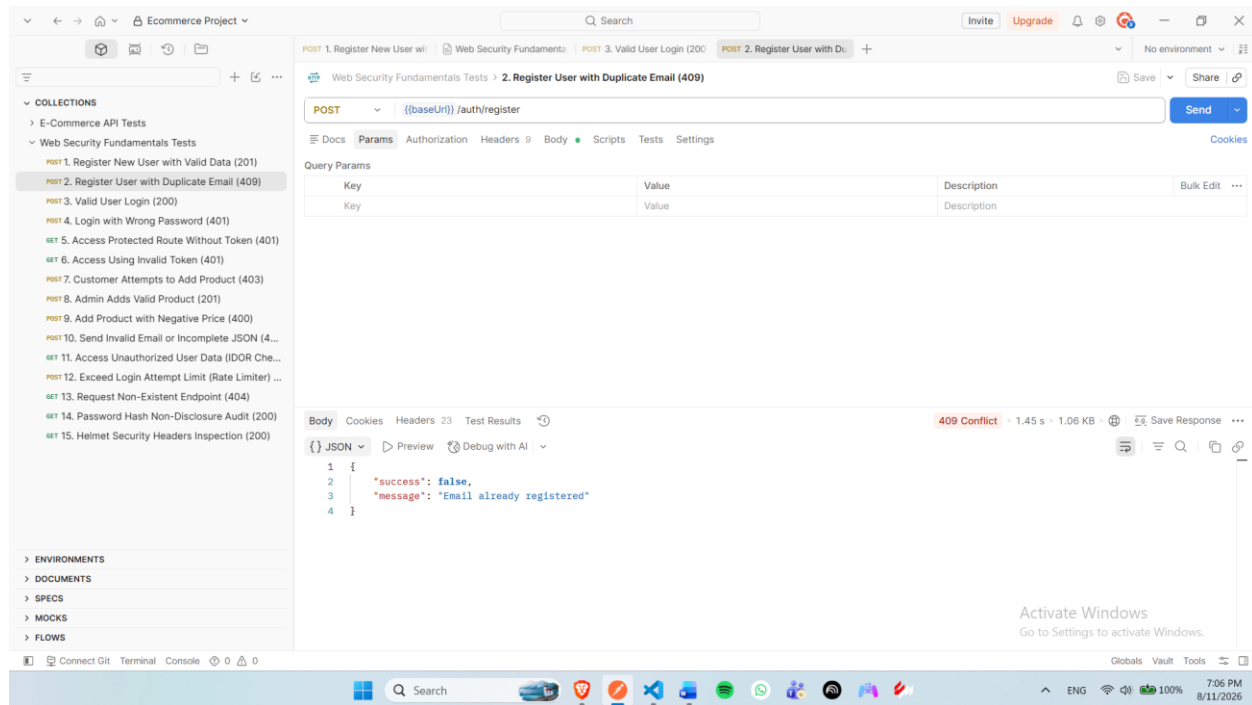
Test 1: Register New User with Valid Data

- **HTTP Method & Route:** POST /api/auth/register
- **Target Result:** 201 Created
- **Description:** Verifies that a new user account can be successfully registered with valid full name, email, password, and role.



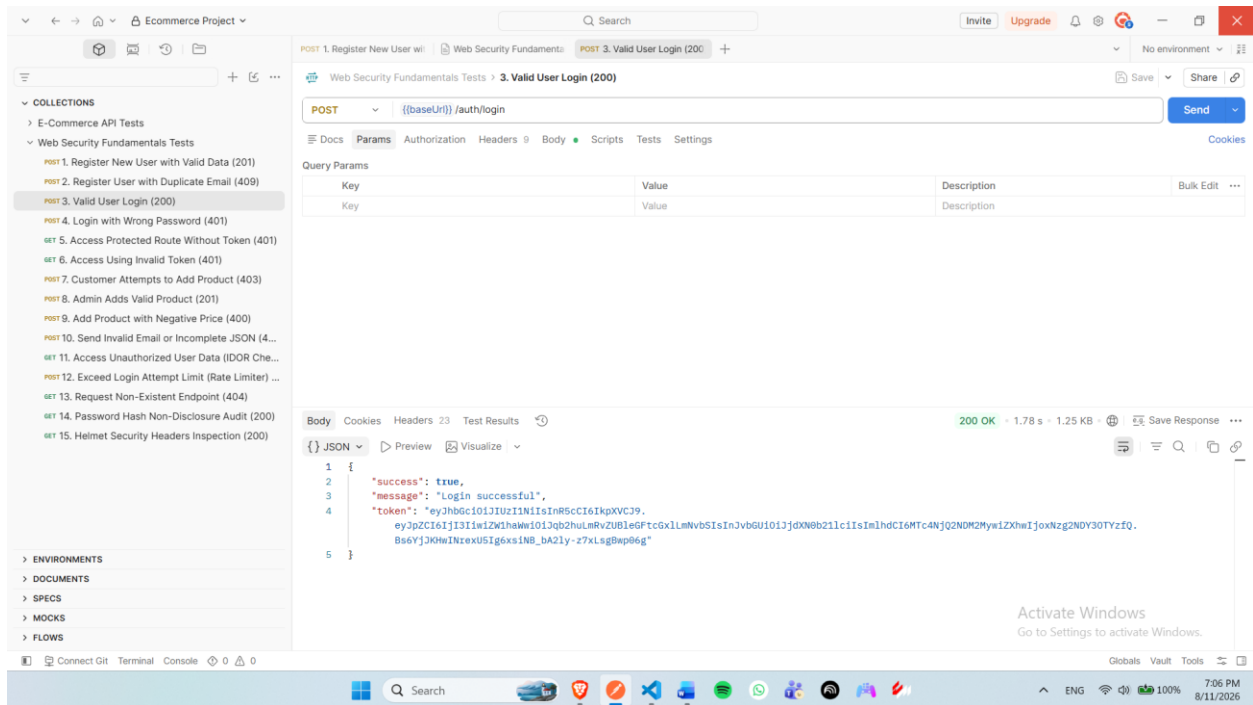
Test 2: Register User with Duplicate Email

- **HTTP Method & Route:** POST /api/auth/register
- **Target Result:** 409 Conflict
- **Description:** Verifies that the system enforces unique email constraints and rejects duplicate registrations cleanly.



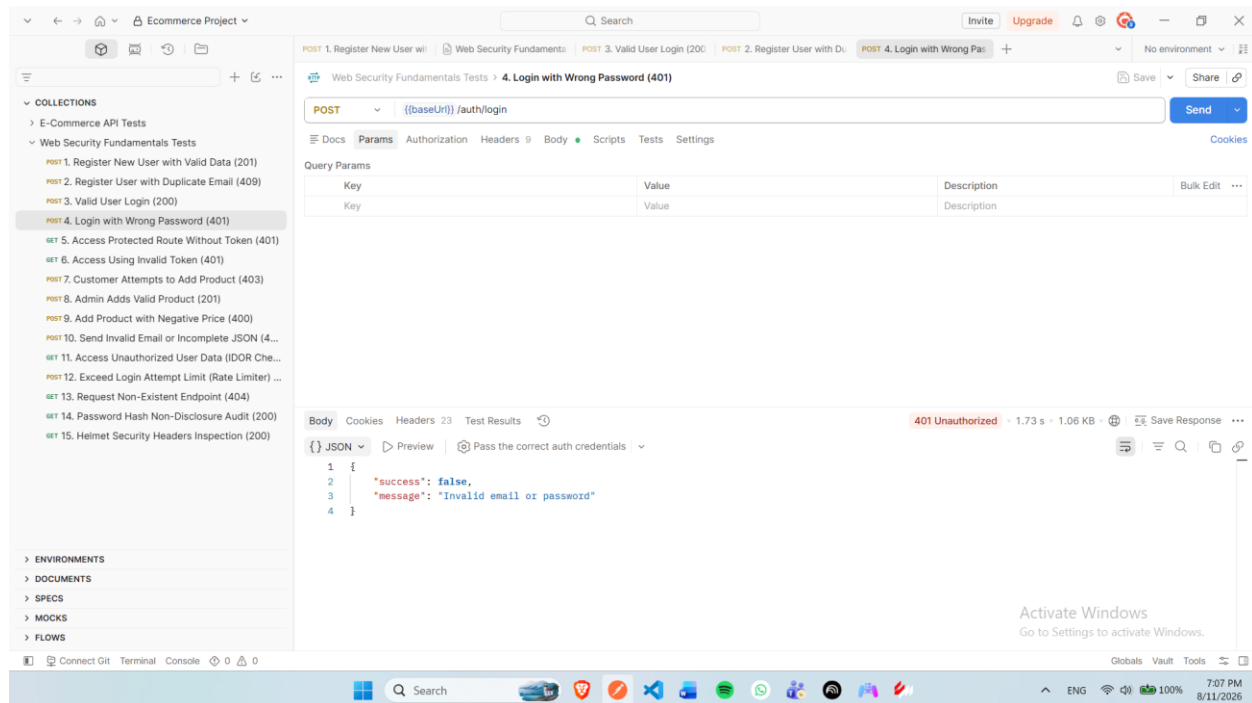
Test 3: Valid User Login

- **HTTP Method & Route:** POST /api/auth/login
- **Target Result:** 200 OK
- **Description:** Verifies that valid user credentials return a signed JWT access token for authorized requests.



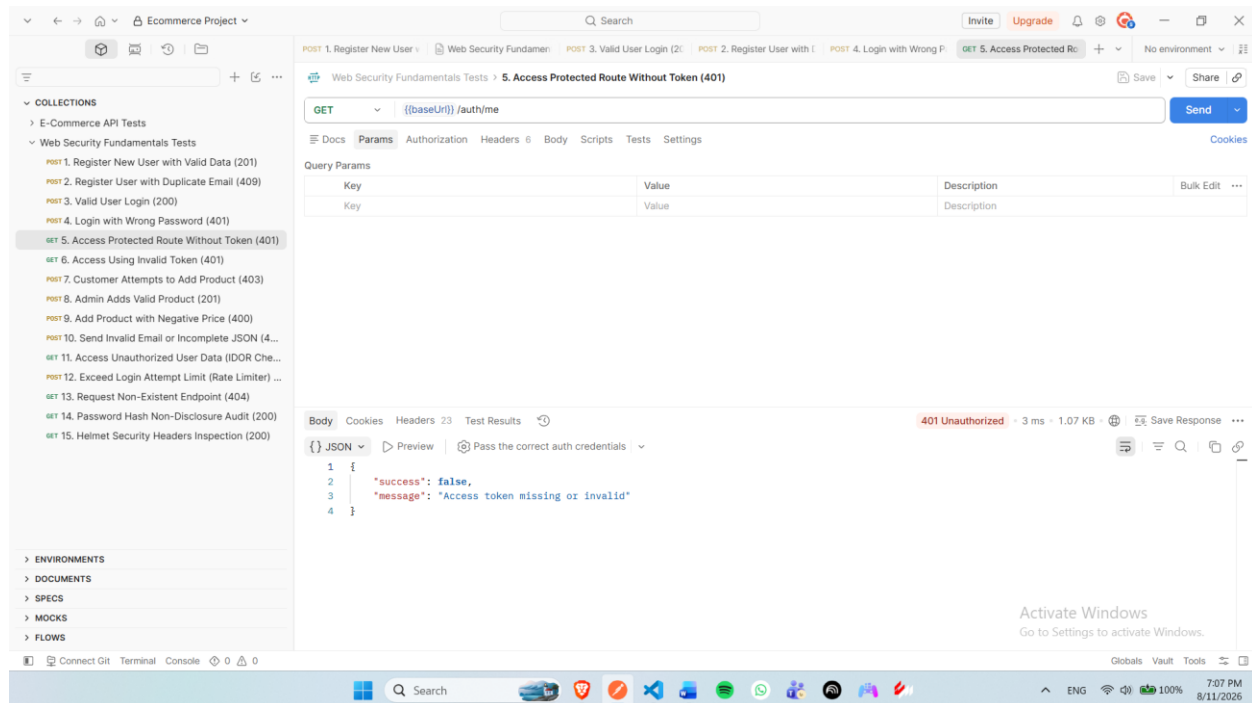
Test 4: Login with Wrong Password

- **HTTP Method & Route:** POST /api/auth/login
- **Target Result:** 401 Unauthorized
- **Description:** Verifies that invalid login attempts are rejected without revealing specific internal account states.



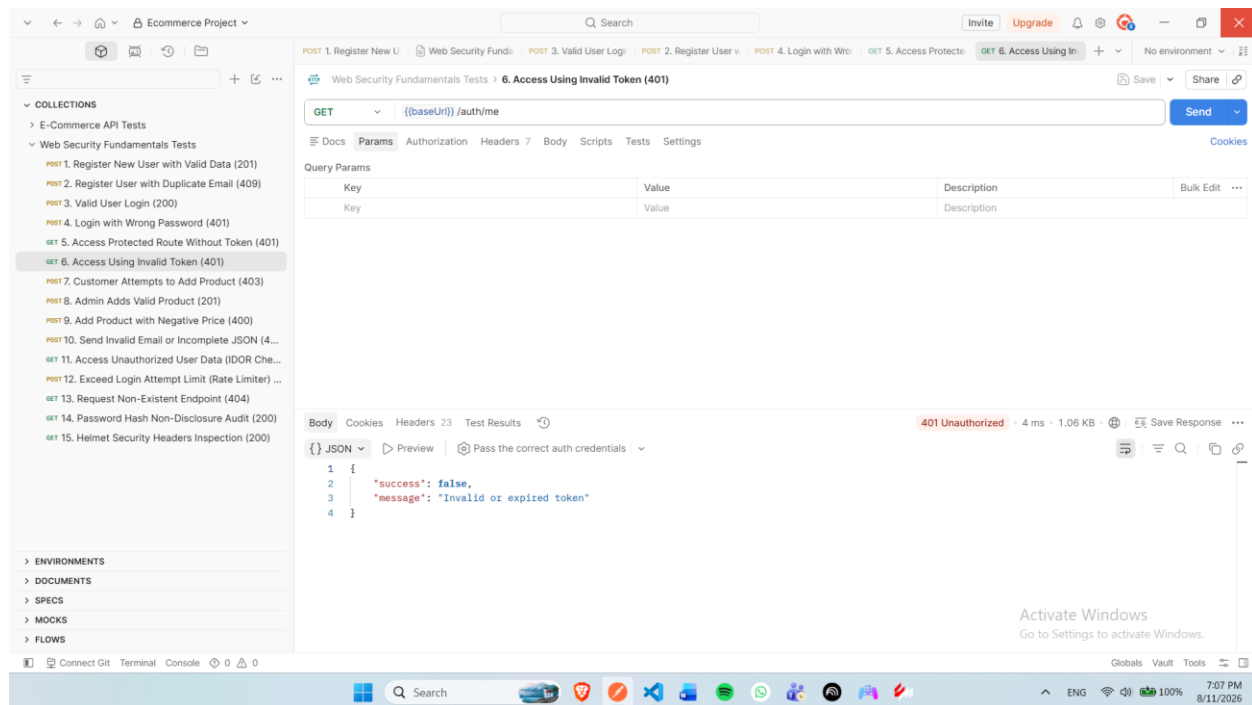
Test 5: Access Protected Route Without Token

- **HTTP Method & Route:** GET /api/auth/me
- **Target Result:** 401 Unauthorized
- **Description:** Verifies that unauthenticated requests lacking a Bearer token in the Authorization header are blocked.



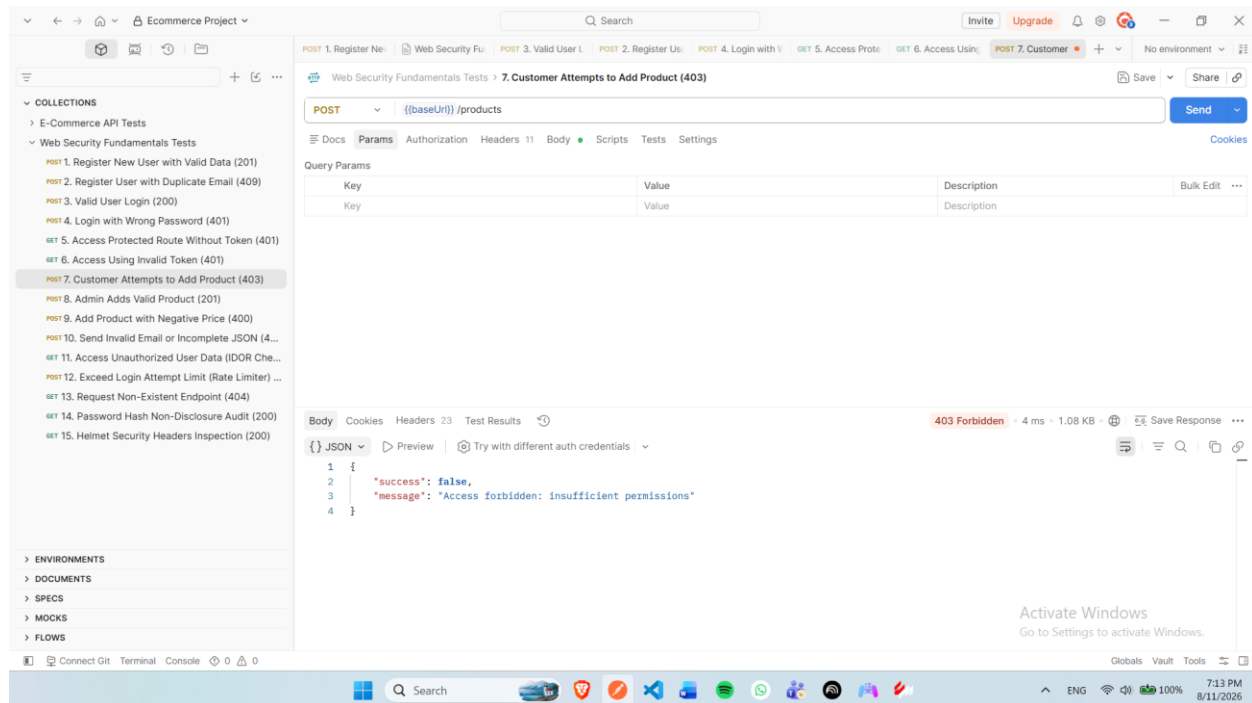
Test 6: Access Using Invalid Token

- **HTTP Method & Route:** GET /api/auth/me
- **Target Result:** 401 Unauthorized
- **Description:** Verifies that tampered or malformed JWT tokens are rejected by the authentication middleware.



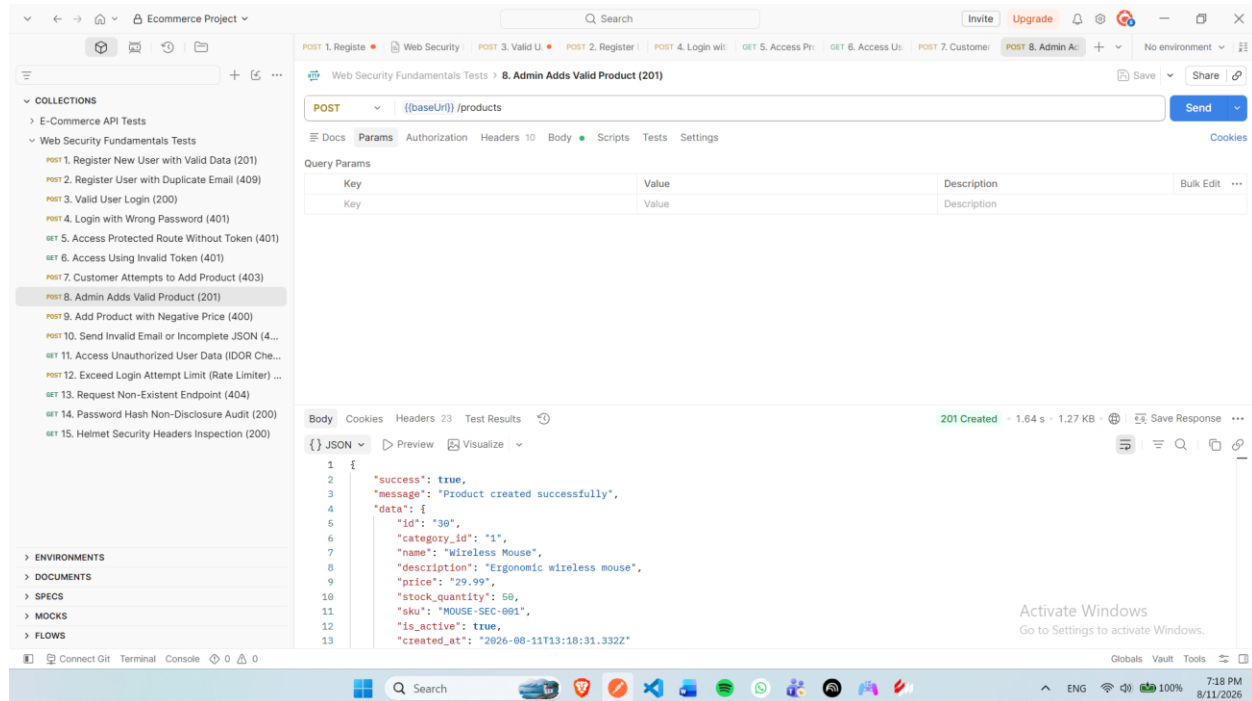
Test 7: Customer Attempts to Add Product

- **HTTP Method & Route:** POST /api/products
- **Target Result:** 403 Forbidden
- **Description:** Verifies Role-Based Access Control (RBAC) by confirming that standard customer tokens are denied admin privileges.



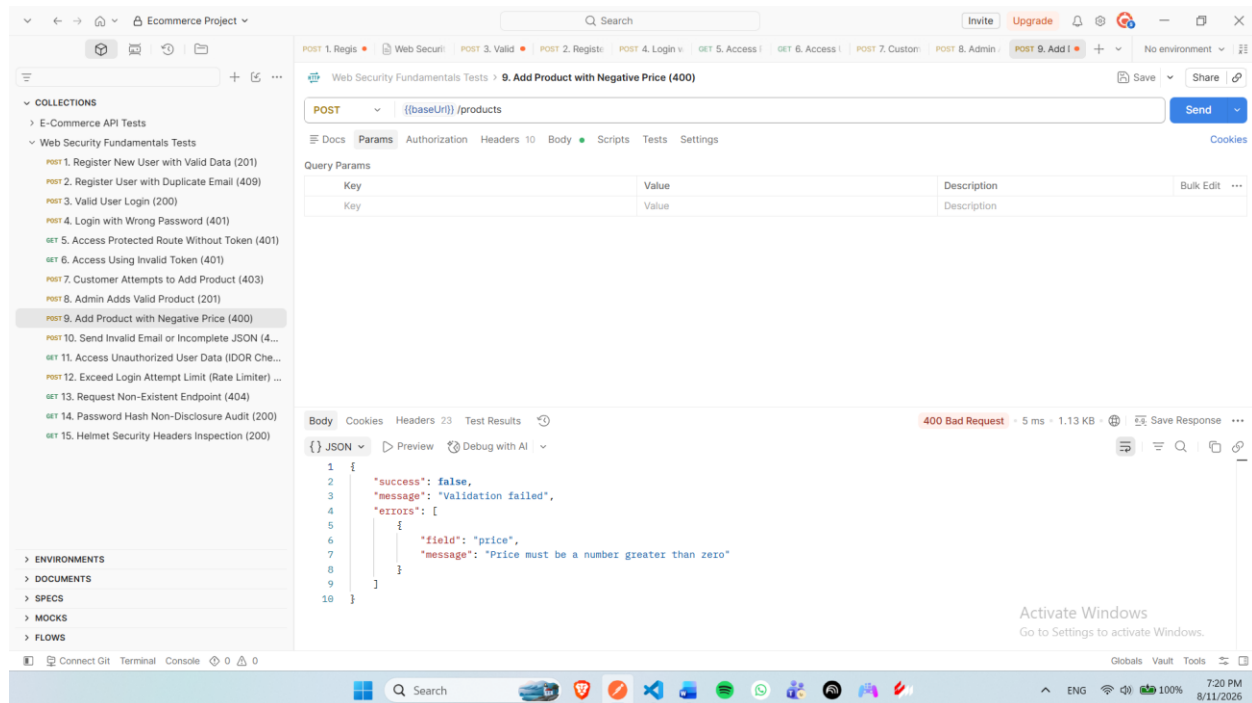
Test 8: Admin Adds Valid Product

- **HTTP Method & Route:** POST /api/products
- **Target Result:** 201 Created
- **Description:** Verifies that an authenticated user with admin privileges can successfully add new products.



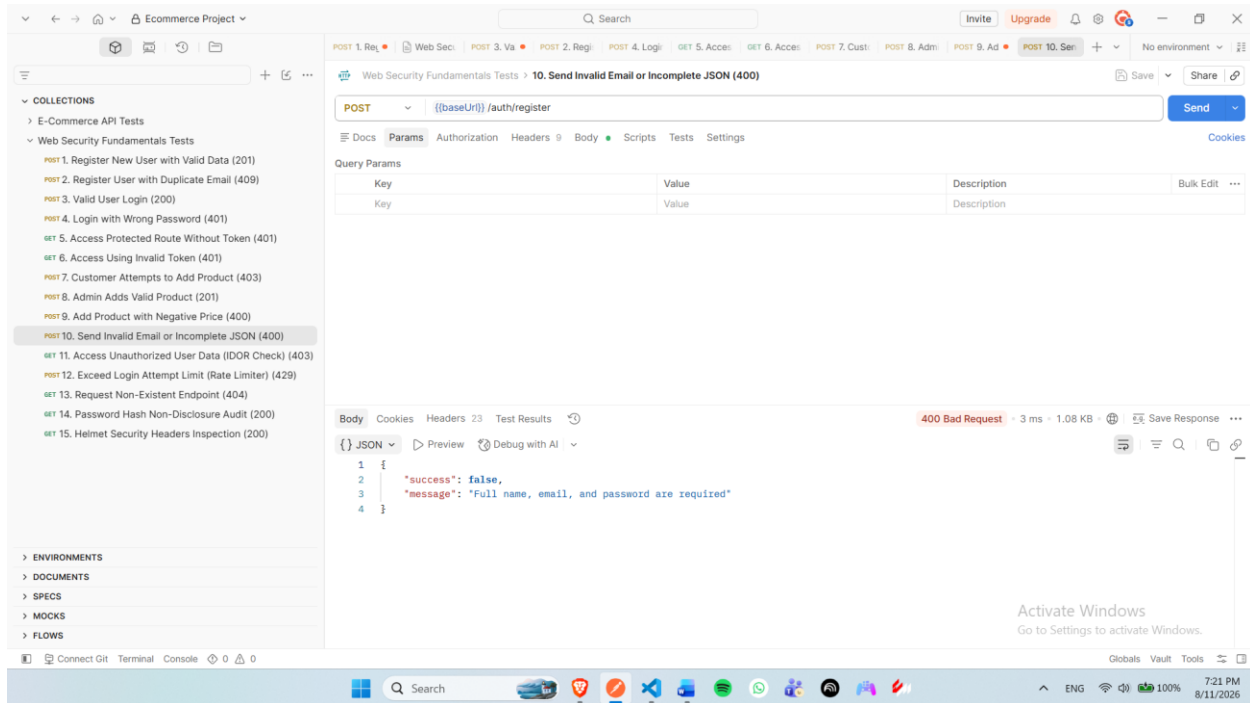
Test 9: Add Product with Negative Price

- **HTTP Method & Route:** POST `/api/products`
- **Target Result:** 400 Bad Request
- **Description:** Verifies that input validation middleware catches invalid numerical values (e.g., negative prices) prior to database query execution.



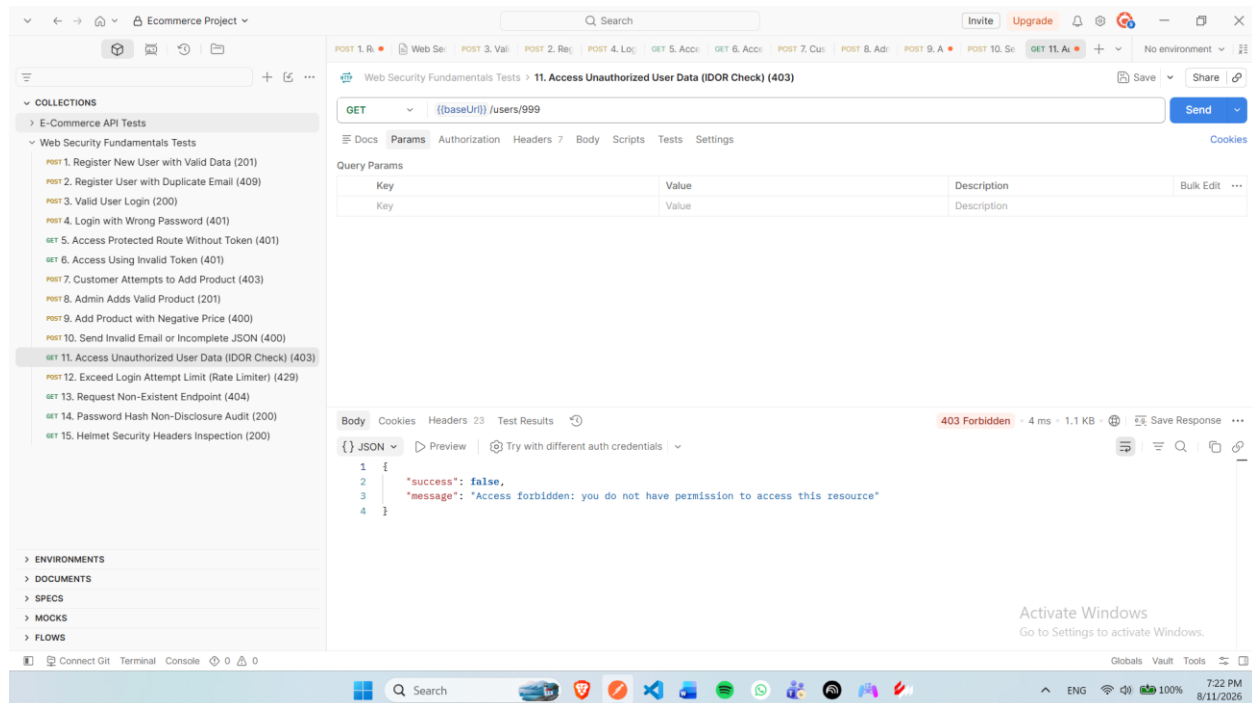
Test 10: Send Invalid Email or Incomplete JSON

- **HTTP Method & Route:** POST /api/auth/register (or POST /api/products)
- **Target Result:** 400 Bad Request
- **Description:** Verifies that malformed email formats or payloads missing mandatory JSON fields return structured validation error messages.



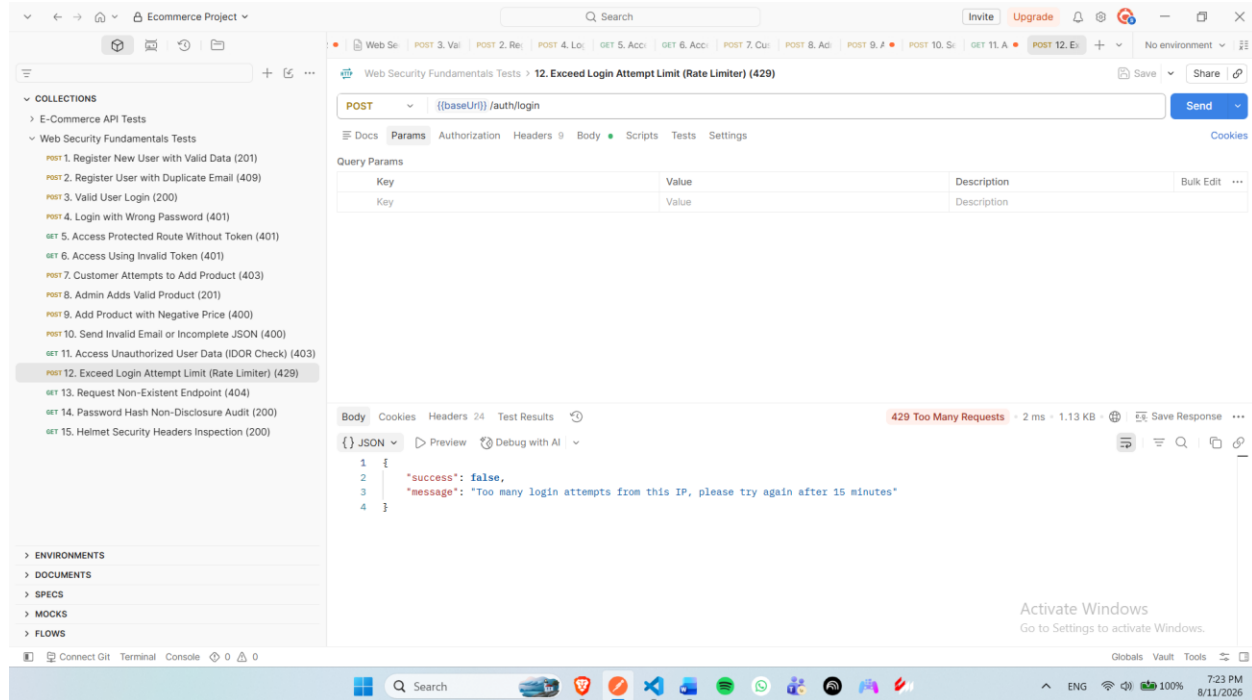
Test 11: Access Unauthorized User Data (IDOR Check)

- **HTTP Method & Route:** GET /api/users/:id
- **Target Result:** 403 Forbidden (or 404 Not Found)
- **Description:** Verifies Insecure Direct Object Reference (IDOR) protection by confirming customers cannot access profile data belonging to another user ID.



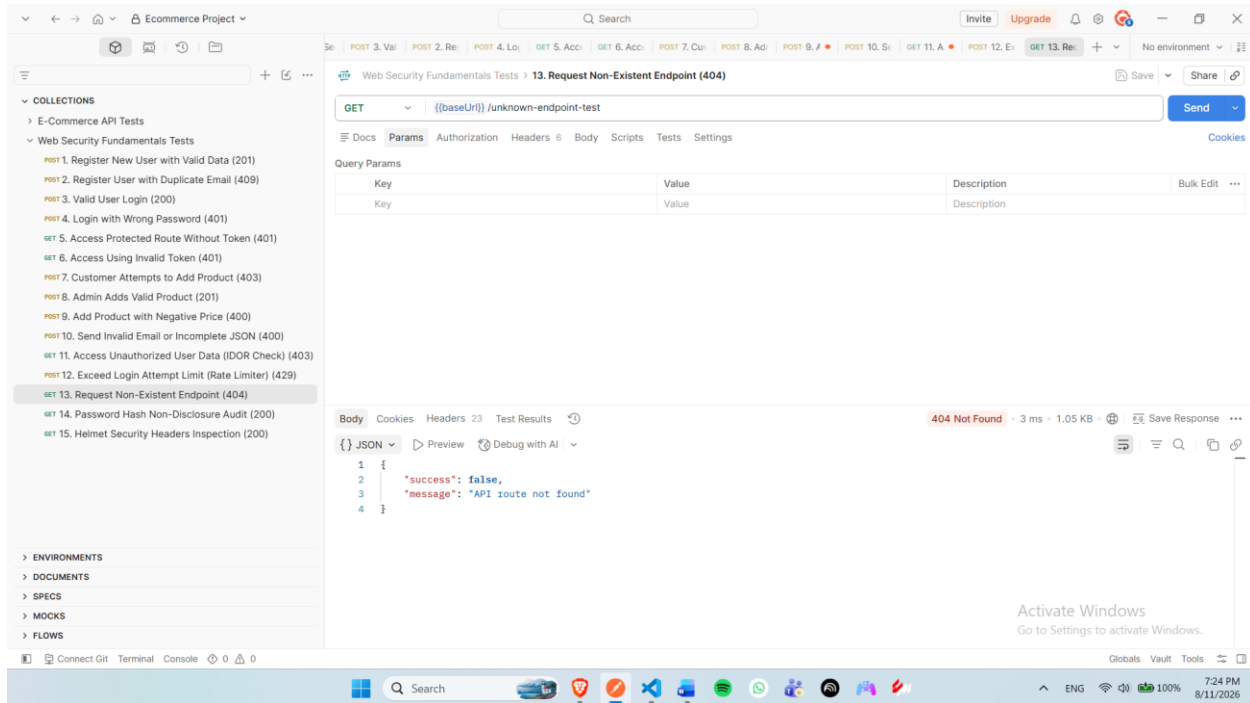
Test 12: Exceed Login Attempt Limit (Rate Limiter)

- **HTTP Method & Route:** POST /api/auth/login
- **Target Result:** 429 Too Many Requests
- **Description:** Verifies brute-force mitigation by making multiple rapid login requests to confirm IP throttling triggers after 5 failed attempts.



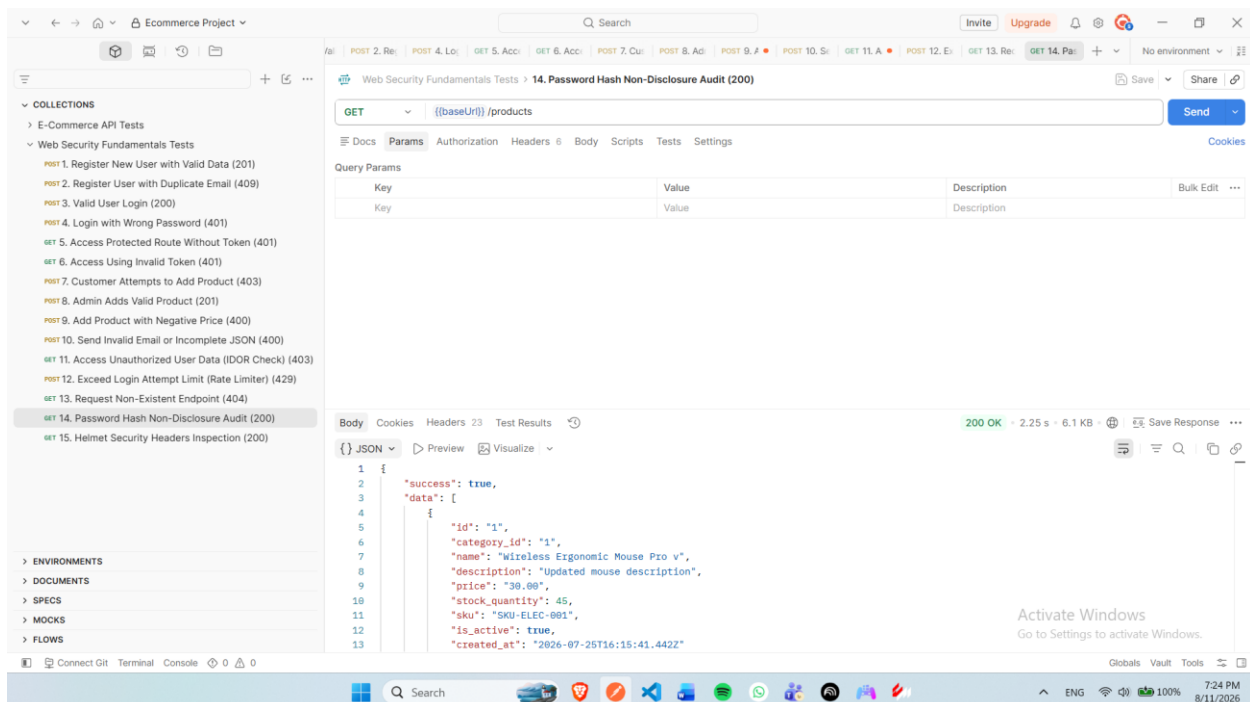
Test 13: Request Non-Existent Endpoint

- **HTTP Method & Route:** GET /api/unknown-endpoint-test
- **Target Result:** 404 Not Found
- **Description:** Verifies that non-existent API routes return a standardized JSON error message via central 404 middleware.



Test 14: Password Hash Non-Disclosure Audit

- **HTTP Method & Route:** GET /api/auth/me or GET /api/users
- **Target Result:** 200 OK (Without password_hash field)
- **Description:** Verifies that sensitive credential hashes (password or password_hash) are completely omitted from response payloads.



Test 15: Helmet Security Headers Inspection

- **HTTP Method & Route:** Any active API Endpoint (Headers Tab)
- **Target Result:** Security Headers Present
- **Description:** Inspects response HTTP headers to confirm Helmet protections (X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security) are active.

The screenshot displays the Postman interface with a GET request to `{{baseUrl}}/products` executed successfully, returning a 200 OK status. The response body is a JSON object representing a product.

Query Params Table:

Key	Value	Description	Bulk Edit
Key	Value	Description	

Response Body (JSON):

```
1 {
2   "success": true,
3   "data": {
4     "id": "1",
5     "category_id": "1",
6     "name": "Wireless Ergonomic Mouse Pro v",
7     "description": "Updated mouse description",
8     "price": "39.99",
9     "stock_quantity": 45,
10    "sku": "SKU-ELEC-001",
11    "is_active": true,
12    "created_at": "2026-07-25T16:15:41.442Z"
13  }
14 }
```

The left sidebar shows a collection of tests under "Web Security Fundamentals Tests", including "15. Helmet Security Headers Inspection (200)".